

Map之一

使用Map集合

北京理工大学计算机学院
金旭亮





Map概述

Map概述



Map集合给每个元素定义了一个Key，通过Key来访问对象。



加入到集合中的每个对象的Key应该不一样，否则，后加入的会覆盖掉前面加入的元素。



Key



Value



当一个元素已经被加入到Map中之后，不要修改它所对应的Key（对象的属性值），这个原则，适用于所有的Map！

Map接口的成员

```

> I ? Map
  > I ? Entry
    (m) ? size(): int
    (m) ? isEmpty(): boolean
    (m) ? containsKey(Object): boolean
    (m) ? containsValue(Object): boolean
    (m) ? get(Object): V
    (m) ? put(K, V): V
    (m) ? remove(Object): V
    (m) ? putAll(Map<? extends K, ? extends V>): void
    (m) ? clear(): void
    (m) ? keySet(): Set<K>
    (m) ? values(): Collection<V>
    (m) ? entrySet(): Set<Entry<K, V>>
    (m) ? equals(Object): boolean ↑Object
    (m) ? hashCode(): int ↑Object
    (m) ? getOrDefault(Object, V): V
    (m) ? forEach(BiConsumer<? super K, ? super V>): void

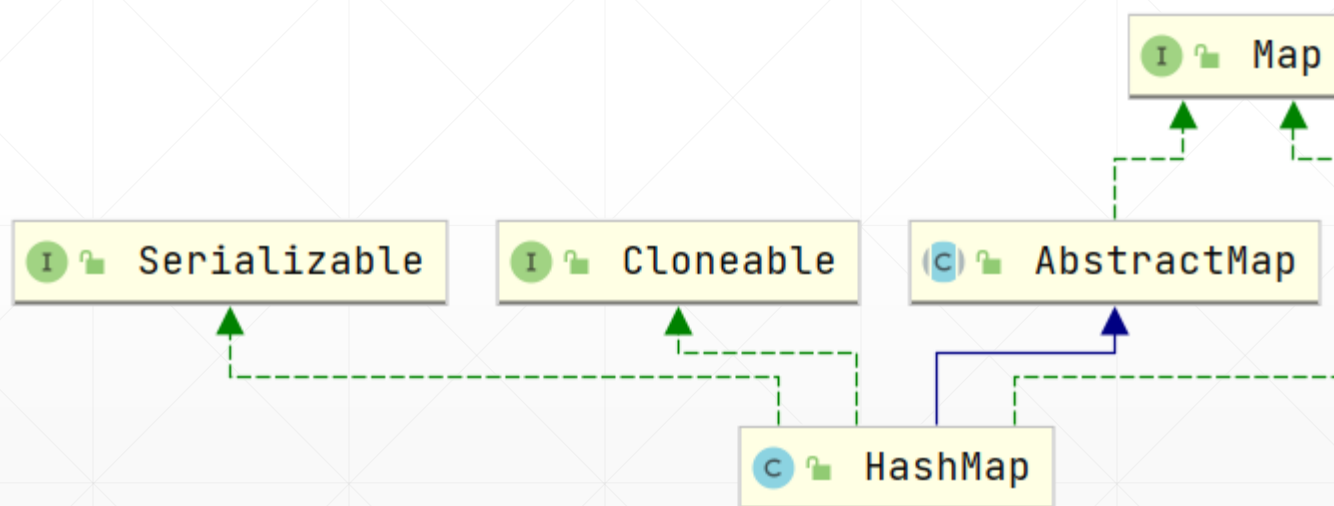
```

- m replaceAll(BiFunction<? super K, ? super V, ? extends V>): void
- m putIfAbsent(K, V): V
- m remove(Object, Object): boolean
- m replace(K, V, V): boolean
- m replace(K, V): V
- m computeIfAbsent(K, Function<? super K, ? extends V>): V
- m computeIfPresent(K, BiFunction<? super K, ? super V, ? extends V>): V
- m compute(K, BiFunction<? super K, ? super V, ? extends V>): V
- m merge(K, V, BiFunction<? super V, ? super V, ? extends V>): V
- m of(): Map<K, V>
- m of(K, V): Map<K, V>
- m of(K, V, K, V): Map<K, V>
- m of(K, V, K, V, K, V): Map<K, V>
- m of(K, V, K, V, K, V, K, ...): Map<K, V>
- m of(K, V, K, V, K, V, K, ...): Map<K, V>
- m of(K, V, K, V, K, V, K, ...): Map<K, V>
- m of(K, V, K, V, K, V, K, ...): Map<K, V>
- m of(K, V, K, V, K, V, K, ...): Map<K, V>
- m of(K, V, K, V, K, V, K, ...): Map<K, V>
- m of(K, V, K, V, K, V, K, ...): Map<K, V>
- m ofEntries(Entry<? extends K, ? extends V>...): Map<K, V>
- m entry(K, V): Entry<K, V>
- m copyOf(Map<? extends K, ? extends V>): Map<K, V>

注意：Map并没有派生自Collection！

HashMap

在开发中最常用的是HashMap<K, V>，它派生自AbstractMap，实现了Map接口，JDK 8引入Lambda特性之后，又给它增强了新的功能。



向HashMap中追加元素时，使用元素（对象）的hashCode作为“Key”。

向Map中添加/移除元素

```
V put(K key, V value)
```

```
void putAll(Map<? extends K, ? extends V> values)
```

加单个元素用put，加多个元素用putAll

```
V remove(Object key)
```

```
void clear()
```

要移除某个元素，需要知道它的key，要清空整个Map，调用clear()方法。

提取和检索元素



要提取元素，需要知道它的key。

```
V get(Object key)
```



可以向Map询问：“某个Key，或者是某个Value，是否在你这儿？”

```
boolean containsKey(Object key)  
boolean containsValue(Object value)
```

了解集合中有多少个元素?

```
int size()
```

```
boolean isEmpty()
```

通过size方法知道当前集合中有多少个元素，使用isEmpty方法判断此集合是否为空（即包容0个元素）



HashMap使用速成

HashMap的基本用法-创建并添加元素

HashMap是最常用的Map，本讲提供了一个UseHashMap示例，通过它可掌握HashMap的基本使用方法：

```
//创建一个示例用的HashMap<String,Integer>
private static HashMap<String, Integer> getStringIntegerHashMap() {
    //创建HashMap对象
    HashMap<String, Integer> hashMap = new HashMap<>();
    //向其中添加四个元素
    for (int i = 1; i < 5; i++) {
        hashMap.put("Key" + i, i);
    }
    //输出: {Key2=2, Key1=1, Key4=4, Key3=3}
    System.out.println(hashMap);
    return hashMap;
}
```

示例UseHashMap.java

HashMap的遍历-1

对HashMap的只读遍历主要是通过其Key集合实现的。这里再次强调，**foreach**循环中不允许移除元素。

基于KeySet遍历HashMap<String, Integer>

```
Set<String> keySet = hashMap.keySet();
for (String key : keySet) {
    System.out.println(key + ":" + hashMap.get(key));
    // 循环遍历过程中, 不允许移除元素, 否则,
    // 引发ConcurrentModificationException
    // if(hashMap.get(key) % 2!=0){
    // hashMap.remove(key);
    // }
}
```

代码摘自visitHashMap()方法

HashMap的遍历-2

如果在遍历过程中需要移除元素，可以通过entrySet生成一个Iterator，然后调用Iterator的remove方法实现这一目的。

```
Set<Map.Entry<String, Integer>> entrySet = hashMap.entrySet();
Iterator<Map.Entry<String, Integer>> itor = entrySet.iterator();
while (itor.hasNext()) {
    Map.Entry<String, Integer> entry = itor.next();
    if (entry.getValue() % 2 != 0) {
        itor.remove(); //移除元素
        System.out.println(entry.getKey() + ":" +
            entry.getValue() + "已被移除");
    } else {
        System.out.println(entry.getKey() + ":"
            + entry.getValue());
    }
}
```

使用Iterator遍历并移除元素

HashMap的遍历-3

Java 8引入了Lambda表达式，所以，可以使用它来简化遍历Map的代码：

```
hashMap.forEach((k, v) ->  
    System.out.println(k + " -> " + v));
```



实际开发中，推荐使用这种方式遍历Map。

如果重复添加相同Key的元素.....

```
private static void addElementToHashMap(HashMap<String,Integer> hashMap) {  
  
    System.out.println("\n原始集合中的元素");  
    //输出: {Key2=2, Key1=1, Key4=4, Key3=3}  
    System.out.println(hashMap);  
    //可以多次向HashMap中追加相同Key的元素, “后来者居上”  
    hashMap.put("Key2", 100);  
    System.out.println("\n修改后的集合元素: ");  
    //输出: {Key2=100, Key1=1, Key4=4, Key3=3}  
    System.out.println(hashMap);  
}
```



注意: HashMap遍历输出所有元素的顺序, 不等于元素加入时的初始顺序。



访问Map中单个元素的方法

```
private static void visitSingleElementInMap(HashMap<String, Integer> hashMap) {  
    //访问元素需要指定key  
    // int value = hashMap.get("Not Existed Key");  
    //如果指定的key不存在, 以下这句将引发NullPointerException,  
    //因为get()返回一个null, 而程序又尝试调用Integer.intValueOf()方法。  
    //System.out.println(value);  
    //为避免抛出异常, 可以在Key不存在时, 返回一个默认值  
    int value = hashMap.getOrDefault(key: "Not Existed Key", defaultValue: -1);  
    //输出: -1  
    System.out.println(value);  
  
    //当指定的Key不存在时, 使用Lambda表达式生成一个!  
    int ranValue = hashMap.computeIfAbsent(key: "Random Number",  
        num -> new Random().nextInt(bound: 100));  
    //某次输出: 59  
    System.out.println(ranValue);  
    //此时, Map中的内容为: {Key2=100, Key1=1, Key4=4, Key3=3, Random Number=59}  
    System.out.println(hashMap);  
}
```

这里用到了Lambda表达式

替换Map中元素的方法

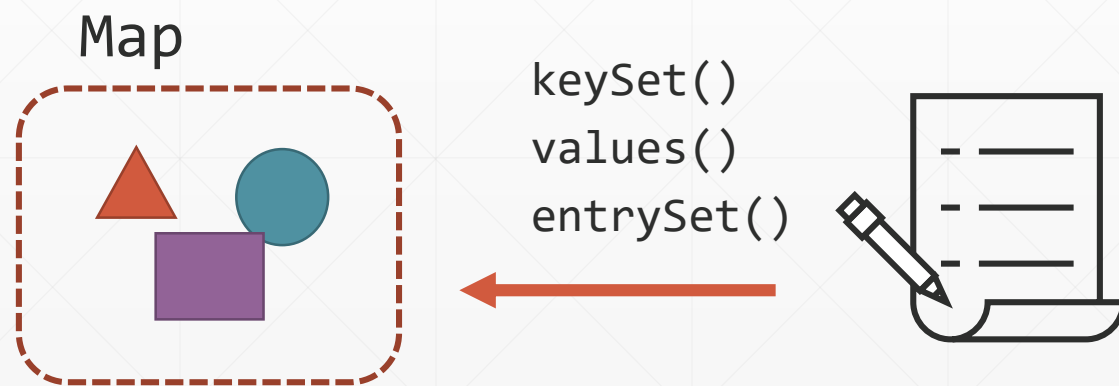
```
private static void replaceElementInMap(HashMap<String, Integer> hashMap) {  
    int oldValue = hashMap.replace("Key1", 1000);  
    //输出: oldValue=1,newValue=1000  
    System.out.println("oldValue=" + oldValue + ",newValue=" + hashMap.get("Key1"));  
    //replace方法的另一个重载, 要求Key和oldValue值都匹配, 才更新其值  
    boolean replaceResult = hashMap.replace(key: "Key1", oldValue: 100, newValue: 10000);  
    //输出: false  
    System.out.println(replaceResult);  
    //这次操作应该成功, 因为Key1当前对应的Value确实是1000  
    replaceResult = hashMap.replace(key: "Key1", oldValue: 1000, newValue: 10000);  
    //输出: true  
    System.out.println(replaceResult);  
    //输出: {Key2=100, Key1=10000, Key4=4, Key3=3, Random Number=78}  
    System.out.println(hashMap);  
    //将所有的元素值全部翻倍  
    hashMap.replaceAll((key, currentValue)->currentValue*2);  
    //输出: {Key2=200, Key1=20000, Key4=8, Key3=6, Random Number=124}  
    System.out.println(hashMap);  
}
```

这里用到了Lambda表达式

Map的“视图 (View)”

通过HashMap对象的`keySet()`方法引用其Key的集合，通过其`values()`方法引用其Value的集合。Key和Value之间的对应关系，是通过HashMap内部的一个`Map.Entry`对象集合实现的，HashMap对象的`entrySet()`方法引用这一对象集合。

`keySet`，`values`和`entrySet`，其实，都可以看成是对底层集合元素的一种“映射”，或者称为一种“视图”，通过它来“间接”地操控集合元素。



通过“Map视图”移除元素

```
var myMap = new HashMap<Integer, MyClass>();  
for (int i = 0; i < 5; i++) {  
    myMap.put(i, ModelHelper.createMyClass());  
}  
System.out.println(myMap);
```

```
var values : Collection<MyClass> = myMap.values();  
System.out.println(values);  
//移除了value,原有的集合也移除了对应的元素  
values.remove(values.toArray()[1]);  
System.out.println(values);  
System.out.println(myMap);
```

```
var keys : Set<Integer> = myMap.keySet();  
System.out.println(keys);  
//移除了key,原有的集合也移除了对应的元素  
keys.remove(0: 1);  
System.out.println(keys);  
System.out.println(myMap);
```

```
var entries : Set<Map<K, V>.Entry<Integer, MyClass>>  
    = myMap.entrySet();  
//移除了entry,原有的集合也移除了对应的元素  
entries.remove(entries.toArray()[1]);  
for (var entry : entries) {  
    System.out.println(entry.getKey() + ":"  
        + entry.getValue());  
}
```

HashMap可以使用null作为Key和Value

```
public class NullInHashMap {  
    public static void main(String[] args) {  
        var hashMap = new HashMap<String, String>();  
        //试图将2个key为null的key-value对放入HashMap中——后来者居上  
        hashMap.put(null, "null value1");  
        hashMap.put(null, "null value2");  
        //输出: null value2  
        System.out.println(hashMap.get(null));  
        //将一个value为null的key-value对放入HashMap中  
        hashMap.put("string key", null);  
        //输出: {null=null value2, string key=null}  
        System.out.println(hashMap);  
    }  
}
```

NullInHashMap.java

Map集合对“相等”的约定



Key相等: Map集合中可用作Key的两个对象，必须同时满足以下两个条件，才能给出结论——它们相等。

两个对象的hashCode一致，并且equals方法返回true。



Value相等: 可用作Value的两个对象，只要它的equals方法返回true，就认为这两个对象相等。

两个对象的equals方法返回true

Key如果可变.....

右图所示，定义了一个类，这个类将作为Map中的Key，注意一下，外界可以使用set()方法修改它的值。

```
private static class MutableString {  
    private String value;  
    public MutableString(final String value) {  
        set(value);  
    }  
    public String get() {  
        return value;  
    }  
    public void set(final String value) {  
        Objects.requireNonNull(value);  
        this.value = value;  
    }  
    public boolean equals(final Object o) {  
        if (this == o) return true;  
        if (o == null || getClass() != o.getClass()) return false;  
        final MutableString that = (MutableString) o;  
        return value.equals(that.value);  
    }  
    public int hashCode() {  
        return value.hashCode();  
    }  
    public String toString() {  
        return value;  
    }  
}
```

MutableHashMapKeys.java

Key可变带来的麻烦~~~

```
public static void main(String[] args) {  
    final Map<MutableString, String> brokenMap = new HashMap<>();  
    final String value = "abc";  
    final MutableString key = new MutableString(value);  
    //把Key值为abc的对象, 加入到HashMap中  
    brokenMap.put(key, value);  
    //输出: abc-->abc  
    System.out.println(key+"-->"+brokenMap.get(key));  
    //输出: Map中的所有元素: {abc=abc}  
    System.out.println("Map中的所有元素: "+brokenMap);  
    //好, 现在修改key的值  
    key.set("def");  
    //输出: def-->null  
    System.out.println(key+"-->"+brokenMap.get(key));  
    //输出: Map中的所有元素: {def=abc}  
    System.out.println("Map中的所有元素: "+brokenMap);  
}
```

现在, 这个元素找不到了, 值为null!

把元素添加到Map中之后, 不要再修改Key的值, 否则, 你就找不到原先加入的元素了!

MutableHashMapKeys.java



其他种类的HashMap

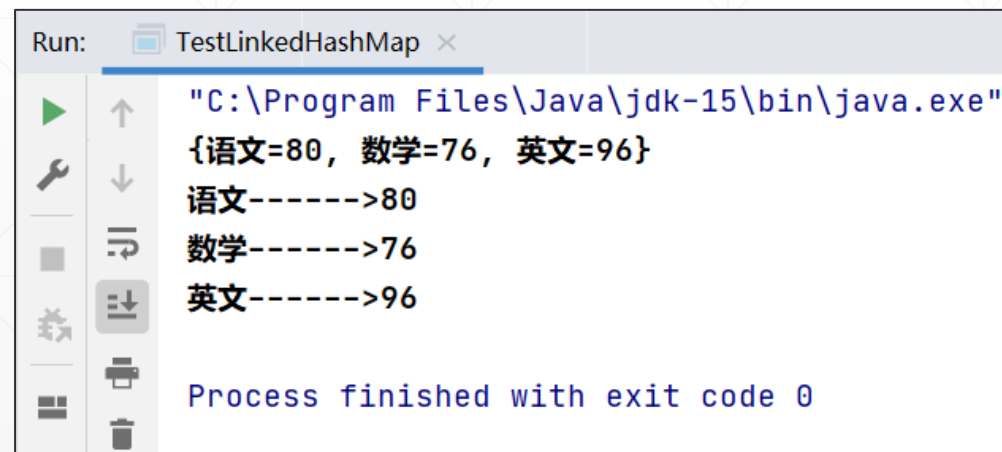
LinkedHashMap简介

LinkedHashMap派生自HashMap，在内部使用双向链表来维护Key-Value对。

```
public class TestLinkedHashMap {  
    public static void main(String[] args) {  
        //试一试, 如果改为HashMap, 看看结果有什么不一样?  
        var scores = new LinkedHashMap<String, Integer>();  
        scores.put("语文", 80);  
        scores.put("数学", 76);  
        scores.put("英文", 96);  
  
        System.out.println(scores);  
        //遍历scores里的所有的key-value对  
        for (Object key : scores.keySet()) {  
            System.out.print(key + "----->");  
            System.out.print(scores.get(key) + "\n");  
        }  
    }  
}
```

示例: TestLinkedHashMap.java

遍历输出集合内容时，是按照元素的加入顺序来输出的。



```
Run: TestLinkedHashMap x  
"C:\Program Files\Java\jdk-15\bin\java.exe"  
{语文=80, 数学=76, 英文=96}  
语文----->80  
数学----->76  
英文----->96  
  
Process finished with exit code 0
```

IdentityHashMap简介

- 它也是一种HashMap，其特殊之处在于：仅当“==”返回true时，两个Key才认为是相等的。

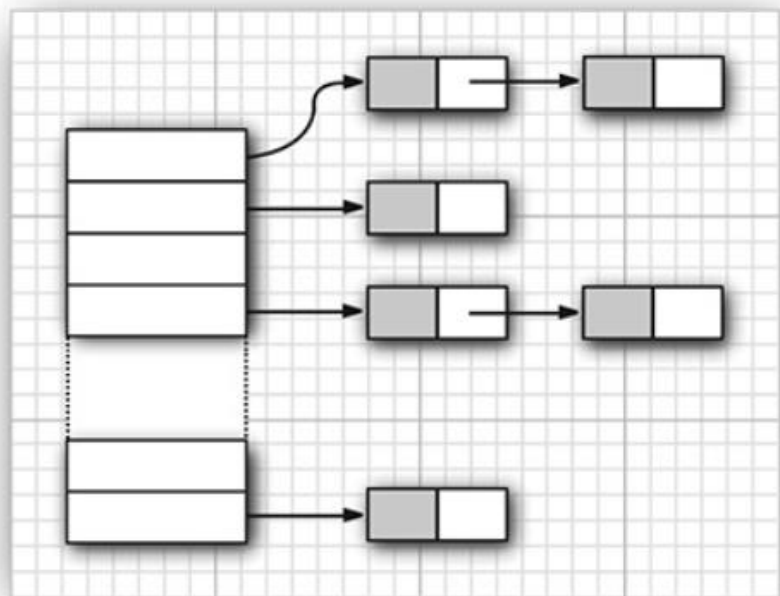
```
public class TestIdentityHashMap {  
    public static void main(String[] args) {  
        var hashMap = new IdentityHashMap<String, Integer>();  
        //下面两行代码将会向IdentityHashMap对象中添加3个key-value对  
        hashMap.put("语文", 89);  
        hashMap.put(new String(original: "语文"), 78);  
        hashMap.put("java", 93);  
        hashMap.put("java", 98);  
        System.out.println(hashMap);  
    }  
}
```

输出：

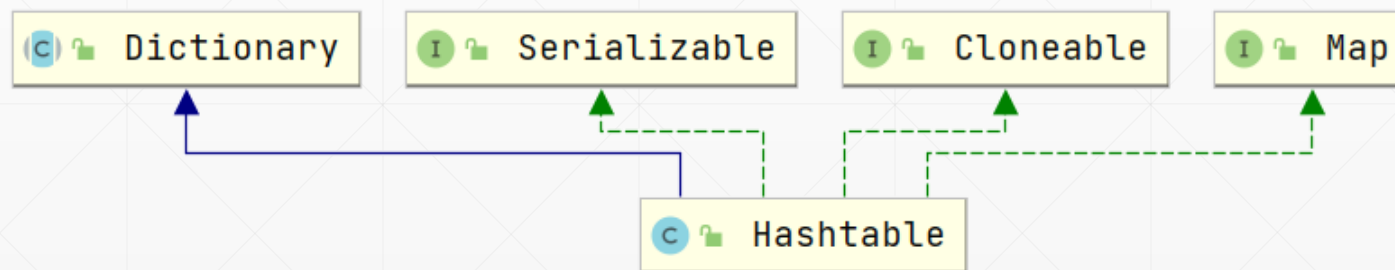
```
{ 语文=89, 语文=78, java=98}
```

示例：TestIdentityHashMap.java

线程安全的HashMap—HashTable



JDK中的HashTable实现了Map接口，它的功能与HashMap基本一样，不同之处在于HashTable是线程安全的。



HashTable示例分析-1

定义好将放到HashTable中的Key和Value类型

```
class MyKey {  
    int count;  
    public MyKey(int count) {  
        this.count = count;  
    }  
    public boolean equals(Object obj) {  
        if (obj == this) {  
            return true;  
        }  
        if (obj != null && obj.getClass() == MyKey.class) {  
            MyKey a = (MyKey) obj;  
            return this.count == a.count;  
        }  
        return false;  
    }  
    public int hashCode() {  
        return this.count;  
    }  
    public String toString() {...}  
}
```

TestHashTable.java

MyKey用作Key，所以，它必须重写基类的equals()和hashCode()方法。作为Value的MyValue类，只需要重写equals()方法就行了。

```
//用于HashTable的Value  
class MyValue {  
    //这里出于演示的目的，让它的equals方法始终返回true  
    @Override  
    public boolean equals(Object obj) {  
        return true;  
    }  
}
```

HashTable示例分析-2

TestHashtable.java

```
//创建一个HashTable对象,  
//此HasTable的Value类型为Object,  
//所以可以接收任何一种类型的数据  
var hashtable = new Hashtable<MyKey,Object>();  
//填充数据  
hashtable.put(new MyKey( count: 60000) , "Hello");  
hashtable.put(new MyKey( count: 87563) , "World");  
hashtable.put(new MyKey( count: 1232) , new MyValue());  
  
//HashTable的toString()方法, 可以显示出它所包容的所有元素  
//{  
// MyKey:60000=Hello,  
// MyKey:87563=World,  
// MyKey:1232=MyValue@4e50df2e  
//}  
System.out.println(hashtable);
```

示例创建了一个演示用的
HashTable集合, 并且向其
中加入了一些测试用的数据。

HashTable示例分析-3

TestHashtable.java

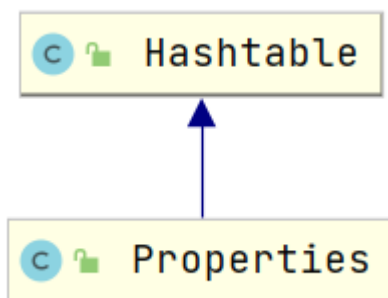
```
//只要两个对象通过equals比较返回true, Hashtable就认为它们是相等的value。  
//因为Hashtable中有一个MyValue对象, 它与任何对象通过equals比较都相等,  
//所以下面这句代码输出true。  
System.out.println(hashtable.containsValue("测试字符串"));  
//只要两个MyKey对象的count属性相等, 它们通过equals比较返回true, 且hashCode相等  
//Hashtable即认为它们是相同的key, 所以下面输出true。  
System.out.println(hashtable.containsKey(new MyKey( count: 87563)));  
//下面语句可以删除最后一个key-value对  
hashtable.remove(new MyKey( count: 1232));  
for (Object key : hashtable.keySet()) {  
    System.out.print(key + "---->");  
    System.out.print(hashtable.get(key) + "\n");  
}
```

示例代码展示了Key和Value的判等特性, 在实际开发中还是很重要的, 必须理解和把握。

示例输出

```
true  
true  
MyKey:60000---->Hello  
MyKey:87563---->World
```

Properties简介



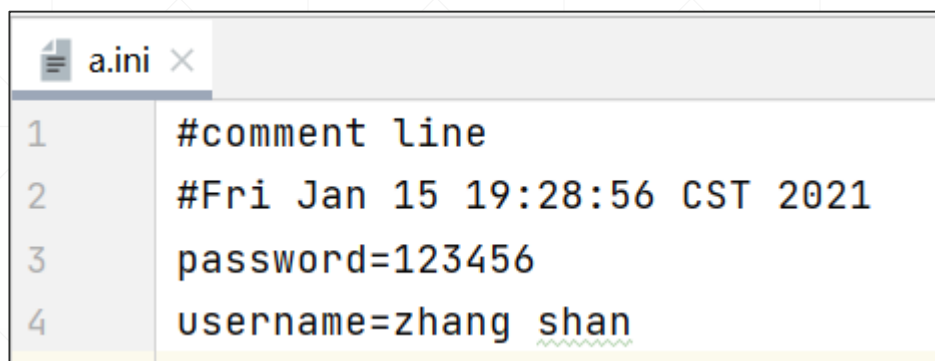
Properties 类派生自 Hashtable，可以方便地处理“属性-值”对，并且可以很方便地将其保存到文件中，在编程中还是比较有用的。

```
public class TestProperties {
    public static void main(String[] args) throws Exception {
        var props = new Properties();
        //向Properties中增加属性
        props.setProperty("username" , "zhang shan");
        props.setProperty("password" , "123456");
        //将Properties中的属性保存到a.ini文件中
        props.store(new FileOutputStream( name: "a.ini") ,
            comments: "comment line");
        //新建一个Properties对象
        var props2 = new Properties();
        //向Properties中增加属性
        props2.setProperty("gender" , "male");
        //将a.ini文件中的属性名-属性值追加到props2中
        props2.load(new FileInputStream( name: "a.ini") );
        //输出: {password=123456, gender=male, username=zhang shan}
        System.out.println(props2);
    }
}
```

TestProperties.java

示例的运行结果

生成的.ini文件内容:

A screenshot of a text editor window titled 'a.ini'. The editor shows four lines of text: line 1 is '#comment line', line 2 is '#Fri Jan 15 19:28:56 CST 2021', line 3 is 'password=123456', and line 4 is 'username=zhang shan'. The text is in a monospaced font, and the line numbers are visible on the left side of the editor.

```
1 #comment line
2 #Fri Jan 15 19:28:56 CST 2021
3 password=123456
4 username=zhang shan
```

运行时的控制台输出:

```
{password=123456, gender=male, username=zhang shan}
```

小结



Map是一种常用的数据结构，用于保存“数值对 (Key-Value)”，也称“字典”。

JDK中最常用的实现就是HashMap<K,V>，大多数场景下，使用它也就足够了

使用Map时，注意其Key最好选中只读的对象。



课后作业

编写一个程序，让用户输入一段英文，程序使用Map统计其中英文单词的出现次数。

参考解答: `WordTypeCount.java`